

TracePilot 多应用页面切换卡顿分析报告

〇、工具链与方法

采集工具链

设备端 (Pixel 6a)

③ Perfetto (--background) ┌
├ atrace: gfx/view/
│ am/wm/res/dalvik
├ ftrace: sched_switch/
│ wakeup + irq/softirq
│ + GPU + block I/O
└ 128MB ring buffer ──┘

④ eBPF loader (60s) ┌
├ tp/sched/sched_switch
├ tp/sched/sched_wakeup
├ tp/irq/irq_handler_*
├ tp/irq/softirq_*
└ events.bin (v2) ──┘

宿主机 (Windows + WSL)

① make bpf && make android
② adb push → 设备

⑤ adb pull trace + events.bin

⑥ trace_processor_shell
-q frame_query.sql
→ frames.txt

⑦ run_analysis.py
→ result_py.json
→ analysis_report.md

步骤	工具	运行环境	产出
① 编译	Makefile + NDK r26b + clang	WSL	tracepilot.bpf.o, tracepilot-aarch64
② 推送	adb push	Windows	设备端 /data/local/tmp/
③ Perfetto	perfetto --background	设备端	perfetto_trace.perfetto-trace
④ eBPF	tracepilot -d 60	设备端	events.bin (sched + sys 事件)
⑤ 拉取	adb pull	Windows	宿主机 output/
⑥ 帧提取	trace_processor_shell	WSL	frames.txt (帧号/预期VSYNC/实际呈现/卡顿标记)
⑦ 分析	run_analysis.py	Windows	result_py.json

数据流

Perfetto trace (33MB)
├ atrace slices → frame_query.sql 解析

```
beginFrame N vsyncIn Xms → 帧号 N, 预期 VSYNC = begin_ts + Xms
presentFrameAndReleaseLayers → 实际呈现时间
incrementJankyFrames → 卡顿判定
输出: 230 帧 (220 jank)
```

└ ftrace sched_switch/wakeup → 设备端 eBPF 也采集, 做交叉验证

└ ftrace GPU/block → 直接 SQL 查询

events.bin (v2, ~300MB)

└ 8,176 条 sched_event (每 96B)
timestamp, event_type, prev/next pid/tid/comm,
prev_state, cpu, wakeup_latency, runnable_delay

└ 932,294 条 system_event (每 32B)
timestamp, event_type, irq_vec, cpu, duration

分析引擎:

events + frames → 时钟对齐 → 帧窗口匹配 → 线程聚合 → p95 计算 → 评分排序

评分公式

$$\text{base_score} = 0.35 \times J + 0.35 \times \log_{1p}(\text{rd_ms}) + 0.15 \times \log_{1p}(\text{wl_ms}) + 0.15 \times \text{UI}$$

J = jank_frame_count / num_jank (该线程参与的卡顿帧占比)
 rd_ms = runnable_delay_p95_ns / 1e6 (被抢占后在队列等待的 p95 毫秒数)
 wl_ms = wakeup_latency_p95_ns / 1e6 (从唤醒到获 CPU 的 p95 毫秒数)
 UI = 1 if comm 含 "RenderThread" 或 ".ui", else 0

sys_ratio = 每帧平均系统开销 (IRQ/SoftIRQ) / 16.67ms
 $\text{score} = \text{base_score} \times (1.0 - \min(\text{sys_ratio}, 0.9))$

设计意图:

- $0.35 \times J$: 出现频次, 线程在所有卡顿帧中的参与度
- $0.35 \times \log_{1p}(\text{rd_ms})$: CPU 抢占强度, rd 越大说明越被调度器冷落
- $0.15 \times \log_{1p}(\text{wl_ms})$: 唤醒响应速度, wl 越大说明等待 CPU 越久
- $0.15 \times \text{UI}$: 渲染相关线程加权 (RenderThread 是帧渲染的关键路径)
- $\log_{1p}$ 压缩极端值 (如 1.38s rd), 避免单一异常值主导排名
- 系统折扣: 扣除 IRQ/SoftIRQ 占用的帧时间, 避免将中断开销误归因给应用线程

噪声过滤

分析时自动排除以下线程:

- **tracepilot**: eBPF 采集器自身
- **shell svc ***: adb shell 进程
- **irq/354-dwc3**: USB 传输中断
- **adbd**: ADB 守护进程

- **UsbFfs-worker**: USB 功能驱动

一、测试场景

项目	说明
测试方式	依次打开 QQ / 微信 / 小红书 / 抖音 / 汽水音乐，来回切换页面
采集时长	60s
atrace 覆盖	apps: * (所有 App)

二、各 App 在 Trace 中的实际占比

2.1 atrace 活动分布

App	atrace slice 数	帧渲染次数	帧平均耗时	帧最大耗时
com.xingin.xhs (小红书)	24,934	584	0.19ms	18.1ms
com.tencent.mobileqq (QQ)	27	0	—	—
com.tencent.mm (微信)	11	0	—	—
com.google.android.apps.scone	17	0	—	—
抖音 / 汽水音乐	未出现在进程表	0	—	—

QQ 和微信的 atrace 数据几乎为零 (27 条和 11 条)，因为 `atrace_apps: "*"` 导致小红书用满缓冲区，其他 App 的 trace 被丢弃。

2.2 帧数据归属

frame_query.sql 提取的 230 帧来自 SurfaceFlinger 全局 VSYNC，**无法区分具体哪个 App**。但从小红书有 584 次 beginFrame、QQ/微信为 0 来看，这 230 帧大概率反映的是小红书的渲染行为。

三、采集概览

指标	数值
总帧数	230
卡顿帧数	220 (95.7%)
eBPF 追踪线程数	8042
ftrace 调度事件	约 44 万次 sched_switch
IRQ/SoftIRQ 系统开销	3372ms (5.6%)

四、Top-20 线程 (eBPF 调度评分)

#	TID	线程名	推断归属	评分	参与帧	RD p95	WL p95
---	-----	-----	------	----	-----	--------	--------

#	TID	线程名	推断归属	评分	参与帧	RD p95	WL p95
1	12649	single-pool-def	小红书	0.369	48/220	1.38s	1.38s
2	2035	SystemUIBg-5	系统 UI	0.294	3/220	355ms	356ms
3	192	thermal_BIG	内核	0.142	178/220	8.6ms	8.8ms
4	537	prng_seeder	内核	0.135	56/220	10.8ms	13.2ms
5	70	ksoftirqd/7	内核	0.113	129/220	0	463ms
6	556	sugov:4	内核	0.105	159/220	0	3.9ms
7	15458	thread_sp_norma	不明	0.100	42/220	0	485ms
8	551	logd.writer	系统	0.093	177/220	0	76ms
9	19046	RenderThread	不明	0.093	220/220	0	16ms
10	7769	Reade2-V15	媒体服务	0.092	220/220	4.1ms	0
11	699	surfaceflinger	系统	0.092	181/220	0	65ms
12	15754	t.mobileqq:tool	QQ	0.091	11/220	0	56ms
13	2546	android.hardwar	系统	0.089	55/220	0	75ms
14	1815	android.imms	系统	0.088	12/220	0	318ms
15	678	mali-cmar-backe	GPU	0.085	123/220	0	76ms
16	656	binder:605_2	系统	0.083	151/220	0	50ms
17	2284	s.nexuslauncher	桌面	0.083	134/220	0	58ms
18	961	android.hardwar	系统	0.082	16/220	0	197ms
19	12625	ReferenceQueueD	不明	0.076	2/220	0	155ms
20	12635	internal_handle	不明	0.076	54/220	0	85ms

五、Perfetto 深度分析

5.1 关键线程 CPU 时间与阻塞状态

线程	CPU(60s)	切换	主要阻塞状态	avg阻塞	max阻塞
RenderThread (19046)	0.13s	813	S(749)/R+(11)/D(2)	0.14ms	3.2ms
binder:605_2 (656)	0.07s	925	S(722)/R+(202)	0.07ms	0.7ms
mali-cmar-backe (678)	0.04s	711	S(682)/D(27)	0.05ms	1.0ms
ksoftirqd/7 (70)	0.04s	847	—	—	—
surfaceflinger (699)	0.01s	177	S(170)/R+(5)	0.03ms	0.5ms

RenderThread 仅使用 0.22% CPU，749 次 S 状态（等 GPU fence），仅 11 次被抢占（R+）。

5.2 GPU 渲染瓶颈

Skia GPU 操作统计:
OpsTask::onExecute 683 次, avg 0.27ms, max 45.6ms
OpsTask::onPrepare 683 次, avg 0.04ms, max 1.5ms
GPU completion fence signaled ~2300 次

Skia OpsTask::onExecute 最长达 45.6ms (约 3 帧周期) , 这是 RenderThread 的主要阻塞源。

5.3 各 App 线程的 runnable_delay

App	线程	RD p95	WL p95	影响
小红书	single-pool-def	1.38s	1.38s	线程池严重抢占 CPU
系统 UI	SystemUIBg-5	355ms	356ms	仅 3 帧, 影响小
内核	thermal_BIG	8.6ms	8.8ms	温控线程持续活动
内核	prng_seeder	10.8ms	13.2ms	随机数种子生成
QQ	t.mobileqq:tool	0	56ms	仅 11 帧, 参与度低

六、问题诊断

问题 1: atrace 覆盖严重不均

App	slice 数	诊断
小红书	24,934	正常
QQ	27	缓冲区被小红书挤占
微信	11	同上
抖音 / 汽水音乐	0	未在 trace 中

根因: atrace_apps: "*" 让所有 App 共享 32MB 缓冲区, 小红书渲染最频繁 (584 帧) , 占满缓冲区, QQ/ 微信的 atrace 数据被丢弃。

问题 2: 小红书线程池严重抢占 CPU

单线程 single-pool-def 的 RD=1.38s, 说明小红书在后台进行密集计算 (可能是推荐算法或内容预加载) , 抢占 CPU 导致其他 App 的渲染线程被排挤。

问题 3: GPU 阻塞是统一的瓶颈

无论哪个 App 的 RenderThread, 都受 GPU 执行时间限制。Skia OpsTask 最长 45.6ms 的 GPU 操作对所有 App 的渲染都有影响。

七、建议

优先级	建议	原因
P0	每个 App 单独采集 ：将 atrace_apps 改为单包名，Perfetto buffer 加到 128MB，各采 30s	避免缓冲区竞争，获取每 App 完整调用栈
P0	采集时关闭其他后台 App	小红书线程池 1.38s rd 直接干扰测试
P1	添加 atrace 类别 <code>res am wm dalvik</code>	覆盖页面切换的核心流程（Activity 启动、View 绘制）
P1	对比「单 App 运行」vs「多 App 切换」的帧 jank 率	量化后台 App 干扰的具体影响
P2	升级 <code>perfetto_config</code> 缓冲区到 128MB	减少 trace 丢弃